

19 Pointers - Zero to Hero questions list

July 18, 2026

1 Pointers in C — Practice Questions (50 Questions with Hints Only)

The questions are arranged from **Beginner** → **Intermediate** → **Advanced**.

2 Beginner Level (1–15)

2.0.1 1. Declare an integer variable and a pointer to it.

Hint: Store the address of the variable using `&`.

2.0.2 2. Print the value of a variable using a pointer.

Hint: Use the dereference operator `*`.

2.0.3 3. Print the address of a variable.

Hint: Use the address-of operator `&`.

2.0.4 4. Print the address stored in a pointer.

Hint: Use `%p` with the pointer variable.

2.0.5 5. Modify a variable's value using a pointer.

Hint: Assign a new value using `*ptr`.

2.0.6 6. Swap two numbers using pointers.

Hint: Pass the addresses of both variables to a function.

2.0.7 7. Find the larger of two numbers using pointers.

Hint: Compare the dereferenced values.

2.0.8 8. Add two numbers using pointers.

Hint: Pass addresses instead of values.

2.0.9 9. Find the square of a number using a pointer.

Hint: Dereference the pointer before calculation.

2.0.10 10. Increment a variable using a pointer.

Hint: Use `(*ptr)++`.

2.0.11 11. Decrement a variable using a pointer.

Hint: Use `(*ptr)--`.

2.0.12 12. Exchange values of two variables using pointers.

Hint: Use a temporary variable.

2.0.13 13. Find the difference between two pointers.

Hint: Point both to elements of the same array.

2.0.14 14. Demonstrate pointer arithmetic.

Hint: Perform `ptr++`, `ptr--`, `ptr + n`, and `ptr - n`.

2.0.15 15. Show the difference between `*ptr++` and `(*ptr)++`.

Hint: Observe operator precedence.

3 Intermediate Level (16–35)

3.0.1 16. Traverse an array using pointers.

Hint: Increment the pointer instead of using indices.

3.0.2 17. Calculate the sum of array elements using pointers.

Hint: Traverse with pointer arithmetic.

3.0.3 18. Find the largest element in an array using pointers.

Hint: Compare dereferenced values.

3.0.4 19. Find the smallest element in an array using pointers.

Hint: Update the minimum value.

3.0.5 20. Reverse an array using pointers.

Hint: Use two pointers from opposite ends.

3.0.6 21. Copy one array into another using pointers.

Hint: Traverse both arrays simultaneously.

3.0.7 22. Count even and odd numbers using pointers.

Hint: Check each dereferenced element.

3.0.8 23. Search for an element using pointers.

Hint: Implement Linear Search.

3.0.9 24. Sort an array using pointers.

Hint: Swap values through pointers.

3.0.10 25. Find the average of array elements using pointers.

Hint: Accumulate values while traversing.

3.0.11 26. Find the length of a string using pointers.

Hint: Stop when '\0' is reached.

3.0.12 27. Copy one string to another using pointers.

Hint: Copy until the null character.

3.0.13 28. Reverse a string using pointers.

Hint: Use start and end pointers.

3.0.14 29. Count vowels in a string using pointers.

Hint: Traverse one character at a time.

3.0.15 30. Compare two strings using pointers.

Hint: Compare characters sequentially.

3.0.16 31. Concatenate two strings using pointers.

Hint: Find the end of the first string.

3.0.17 32. Convert a string to uppercase using pointers.

Hint: Modify characters in place.

3.0.18 33. Convert a string to lowercase using pointers.

Hint: Traverse character by character.

3.0.19 34. Count words in a string using pointers.

Hint: Detect spaces and word boundaries.

3.0.20 35. Find the first occurrence of a character using pointers.

Hint: Traverse until a match is found.

4 Functions and Dynamic Memory (36–45)

4.0.1 36. Pass a pointer to a function and modify the original variable.

Hint: Observe pass-by-reference behavior.

4.0.2 37. Pass an array to a function using pointers.

Hint: Arrays decay into pointers.

4.0.3 38. Return a pointer from a function.

Hint: Do not return the address of a local variable.

4.0.4 39. Dynamically allocate memory for an integer.

Hint: Use `malloc()`.

4.0.5 40. Dynamically allocate memory for an array.

Hint: Allocate `n * sizeof(type)` bytes.

4.0.6 41. Resize dynamically allocated memory.

Hint: Use `realloc()`.

4.0.7 42. Release dynamically allocated memory.

Hint: Use `free()` and set the pointer to `NULL`.

4.0.8 43. Create a dynamic array and calculate its sum.

Hint: Combine `malloc()` with pointer traversal.

4.0.9 44. Create a dynamic string.

Hint: Allocate memory based on user input length.

4.0.10 45. Allocate memory for an array of structures.

Hint: Use `malloc()` with `sizeof(struct)`.

5 Advanced Level (46–50)

5.0.1 46. Create a linked list using pointers.

Hint: Each node contains data and a pointer to the next node.

5.0.2 47. Reverse a singly linked list.

Hint: Use three pointers: previous, current, and next.

5.0.3 48. Implement a stack using pointers (linked list).

Hint: Implement `push()` and `pop()` operations.

5.0.4 49. Implement a queue using pointers (linked list).

Hint: Maintain both front and rear pointers.

5.0.5 50. Build a Dynamic Student Management System using pointers.

Hint: Divide the program into functions such as:

- Add Student

- Display Students
- Search Student
- Update Student
- Delete Student
- Dynamically Allocate/Resize Records
- Free All Allocated Memory Before Exit